UNIVERSIDADE FEDERAL DO CEARÁ

Test Smells em projetos OpenSource

Disciplina: Qualidade de Software
Equipe: Ana Letícia Oliveira Mesquita - 570889, Maria Beatriz Ribeiro de Castro - 565662
Projeto: github.com/trustedsec/hate_crack
Fork: github.com/Beeatriz-Castro/hate_crack/tree/refatoracao-test-smells

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Redundant Print — arquivo tests/test_hashview.py, método test_create_job_workflow (linhas 2118–2182). Eram 11 ocorrências de print() no mesmo método.
Minhas principais dificuldades na remoção do code smell são: O método tinha um bloco if "job_id" in job_result: cujo único conteúdo era um print. Ao remover o print, o if ficaria vazio (erro de sintaxe), então precisei remover o bloco if inteiro junto. Confirmei antes que ele só imprimia e não fazia nenhuma asserção. Fora isso, sem dificuldades.
Usei o CATSPY para: Definição e Refatoração (página “Redundant Print”).
Eu estou usando os seguintes métodos de refatoração para remover o code smell: Remoção das instruções print() desnecessárias, mantendo apenas a asserção (assert job_result is not None), conforme o CATSPY: “remover instruções de impressão desnecessárias e melhorar a eficiência e clareza dos testes”. O teste continuou passando após a mudança.

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Conditional Test Logic — arquivo tests/test_hashview.py, método test_upload_skips_wrong_type_line (linha 998). O teste tinha um if sent is None: no corpo.
Minhas principais dificuldades na remoção do code smell são: Entender que o if não era lógica de teste ramificada, e sim uma normalização de onde o dado enviado estava (parâmetro data= vs posicional). Decidi extrair o condicional para um método auxiliar em vez de só apagá-lo, para não quebrar o teste caso o corpo venha de forma posicional
Usei o CATSPY para: Definição e Refatoração (página “Conditional Test Logic”).
Eu estou usando os seguintes métodos de refatoração para remover o code smell: Extração do bloco condicional para um método separado (_post_body), conforme o CATSPY: “identifique blocos de código condicional complexos em seus métodos de teste e extraia-os para métodos separados”. O teste passou a chamar sent = _post_body(api.session.post) e ficou sem condicional no corpo.

UNIVERSIDADE FEDERAL DO CEARÁ

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Conditional Test Logic — tests/test_hashview.py, test_upload_preserves_non_utf8_plaintext (linha 1026).
Minhas principais dificuldades na remoção do code smell são: nenhuma nova. Mesmo padrão if sent is None do teste anterior; reaproveitei o helper já criado.
Usei o CATSPY para: Definição e Refatoração (página “Conditional Test Logic”).
Eu estou usando os seguintes métodos de refatoração para remover o code smell: Substituí o bloco condicional por uma chamada ao método auxiliar _post_body(api.session.post), extraindo a condicional do corpo do teste (CATSPY: “extraia os blocos condicionais para métodos separados”).

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Conditional Test Logic - tests/test_hashview.py, test_upload_preserves_hex_wrapped_valid_utf8_plaintext (linha 1062).
Minhas principais dificuldades na remoção do code smell são: mesmo padrão if sent is None; reutilizei o helper _post_body.
Usei o CATSPY para: Definição e Refatoração (página “Conditional Test Logic”).
Eu estou usando os seguintes métodos de refatoração para remover o code smell: Substituí o bloco condicional por sent = _post_body(api.session.post), extraindo a condicional do corpo do teste (CATSPY: “extraia os blocos condicionais para métodos separados”).

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Conditional Test Logic - tests/test_hashview.py, test_upload_retries_batch_after_server_rejects_one_plaintext (linha 1115).
Minhas principais dificuldades na remoção do code smell são: o if usava call_args_list[1] (a 2ª chamada). Como o teste garante que houve exatamente 2 chamadas, a 2ª é a última, então o helper _post_body (que lê a última chamada) retorna o corpo correto.
Usei o CATSPY para: Definição e Refatoração (página “Conditional Test Logic”).
Eu estou usando os seguintes métodos de refatoração para remover o code smell: Substituí o bloco condicional por retry_body = _post_body(api.session.post) (CATSPY:

“extraia os blocos condicionais para métodos separados”).

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Verbose Test - tests/test_hashview.py, test_download_left_with_auto_merge (linhas 2343–2426, ~85 linhas).
Minhas principais dificuldades na remoção do code smell são: o teste tinha preparação pesada (mocks de duas respostas, potfile, cwd) seguida de muitas asserções. Precisei mover toda a preparação pra uma @pytest.fixture (left_auto_merge) pra poder dividir sem duplicar o setup.
Usei o CATSPY para: Definição e Refatoração (página “Verbose Test”).
Eu estou usando os seguintes métodos de refatoração para remover o code smell: Dividi o método em 4 testes menores e focados (criação do arquivo, merge das hashes, limpeza dos intermediários, escrita no potfile), com a preparação compartilhada na fixture (CATSPY: “separe o método de teste em métodos menores e mais focados”).

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Verbose Test - tests/test_hashview.py, test_upload_retries_batch_after_server_rejects_one_plaintext (~58 linhas).
Minhas principais dificuldades na remoção do code smell são: o teste capturava stdout (capsys) além das asserções; movi o capsys.readouterr() pra dentro da fixture, que devolve a saída já capturada pros testes.
Usei o CATSPY para: Definição e Refatoração (página “Verbose Test”).
Eu estou usando os seguintes métodos de refatoração para remover o code smell: Dividi em 4 testes focados (dois POSTs, conteúdo do retry, contagem, mensagem na saída) com a preparação numa @pytest.fixture retry_after_reject (CATSPY: “separe o método de teste em métodos menores e mais focados”).

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Verbose Test - tests/test_omen_attack.py, test_builds_correct_command.
Minhas principais dificuldades na remoção do code smell são: o teste checa uma única coisa (o comando) com muitos asserts, então dividir não fazia sentido; extraí a preparação pra uma fixture.
Usei o CATSPY para: Verbose Test
Eu estou usando os seguintes métodos de refatoração para remover o code smell:

movi toda a montagem/execução pra @pytest.fixture omen_train_cmd, deixando o teste só com as asserções (CATSPY: reduzir o método de teste verboso).

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Verbose Test - tests/test_omen_attack.py, test_builds_correct_pipe_commands.
Minhas principais dificuldades na remoção do code smell são: o teste validava dois comandos diferentes (enum e hashcat) no mesmo método; movi a execução pra uma fixture pra poder separar sem duplicar o setup.
Usei o CATSPY para: Verbose Test
Eu estou usando os seguintes métodos de refatoração para remover o code smell: dividi em 3 testes focados (duas chamadas, comando do enum, comando do hashcat) com a preparação na @pytest.fixture omen_pipe.

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Verbose Test - tests/test_config_startup_wiring.py, test_case1_migrates_integration_keys_and_prunes_config_json (38 linhas de código, 21 asserts).
Minhas principais dificuldades na remoção do code smell são: o teste validava 4 coisas distintas (cria .env, poda o config.json, mensagem de saída, recarregar). Movi a migração pra uma fixture pra separar sem repetir o setup.
Usei o CATSPY para: Verbose Test
Eu estou usando os seguintes métodos de refatoração para remover o code smell: dividi em 4 testes focados com a preparação na @pytest.fixture case1_migration.

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Verbose Test - tests/test_ntlm_preprocessing.py, test_full_pipeline_with_filtering (45 linhas, 16 asserts).
Minhas principais dificuldades na remoção do code smell são: era um pipeline sequencial (contar→filtrar→extrair NT→extrair LM). Rodei o pipeline inteiro numa fixture e separei as verificações em testes focados.
Usei o CATSPY para: Verbose Test
Eu estou usando os seguintes métodos de refatoração para remover o code smell: dividi em 5 testes focados com a @pytest.fixture ntlm_pipeline.



UNIVERSIDADE
FEDERAL DO CEARÁ

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Verbose Test — tests/test_omen_attack.py, test_builds_correct_command.
Minhas principais dificuldades na remoção do code smell são: o teste checa uma única coisa (o comando) com muitos asserts, então dividir não fazia sentido; extraí a preparação pra uma fixture.
Usei o CATSPY para: Verbose Test
Eu estou usando os seguintes métodos de refatoração para remover o code smell: movi toda a montagem/execução pra @pytest.fixture omen_train_cmd, deixando o teste só com as asserções (CATSPY: reduzir o método de teste verboso).

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Verbose Test - tests/test_version_check.py, test_run_upgrade_switches_from_dev_to_main_then_upgrades.
Minhas principais dificuldades na remoção do code smell são: o teste verificava várias etapas da sequência de upgrade (ordem das chamadas git, comando final, mensagem). Rodei _run_upgrade numa fixture e separei em testes focados.
Usei o CATSPY para: Verbose Test
Eu estou usando os seguintes métodos de refatoração para remover o code smell: dividi em 6 testes focados com a @pytest.fixture upgrade_dev_to_main.

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Conditional Test Logic - tests/test_main_spoonman.py, test_the_corpus_is_recognized_through_a_symlink_or_a_relative_form..
Minhas principais dificuldades na remoção do code smell são: o teste usava if/elif sobre um parâmetro “style” pra montar cada caso; separei cada ramo num teste próprio, mantendo a asserção comum num helper..
Usei o CATSPY para: Conditional Test Logic (separar cada caso condicional em um teste).
Eu estou usando os seguintes métodos de refatoração para remover o code smell: removi o if/elif, criei a fixture corpus_setup e o helper _assert_alias_recognized, e dividi em 4 testes (symlink, dot_prefix, parent_traversal, case_differing).

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Sleepy Test - tests/test_notify_suppression.py, test_worker_thread_suppression_does_not_leak (linha 66).

UNIVERSIDADE
FEDERAL DO CEARÁ

Minhas principais dificuldades na remoção do code smell são: o time.sleep(0.05) era usado pra sincronizar threads; troquei por dois threading.Event (entered/may_exit) pra deixar a sincronização determinística, sem espera fixa.
Usei o CATSPY para: Sleepy Test.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: removi a pausa explícita e usei eventos — a main amostra o estado só quando o worker confirma que entrou no bloco, e depois o libera. Teste mais confiável e sem risco de flaky.

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Non-Functional Statement - tests/test_version_check.py, test_flag_selects_channel (linha 806).
Minhas principais dificuldades na remoção do code smell são: o pass estava num except SystemExit; como main() sempre sai com SystemExit nesse teste, troquei o try/except ... pass por with pytest.raises(SystemExit), eliminando a instrução sem função.
Usei o CATSPY para: Non-Functional Statement.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: removi o pass (e o try/except manual), usando pytest.raises(SystemExit) - asserção explícita e idiomática.

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Sleepy Test - tests/test_notify_tailer.py, test_seeks_to_eof_on_start (linha 109).
Minhas principais dificuldades na remoção do code smell são: o teste esperava um tempo fixo pra checar que nada disparava; troquei por uma linha-sentinela + _wait_until, que espera só até a varredura rodar.
Usei o CATSPY para: Sleepy Test.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: removi o time.sleep(0.2) e usei o helper _wait_until já existente; a asserção passou a provar que só a linha nova notificou.

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Sleepy Test - tests/test_notify_tailer.py, test_truncation_resets_position (linha 368).
Minhas principais dificuldades na remoção do code smell são: o sleep garantia que o tailer já estava posicionado no fim do arquivo antes de truncar; troquei por _wait_until(lambda: tailer._file_pos == original_size), que espera exatamente esse estado.

Usei o CATSPY para: Sleepy Test.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: removi o time.sleep(0.15), usando o helper _wait_until e o atributo _file_pos do tailer pra sincronizar de forma determinística.

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Sleepy Test - tests/test_progress_spinner.py, test_spinner_yields_handle_and_renders_detail (linha 176).
Minhas principais dificuldades na remoção do code smell são: o sleep esperava o thread de repaint; criei um helper _wait_until e esperei até o texto do detalhe aparecer no buffer, em vez de tempo fixo.
Usei o CATSPY para: Sleepy Test.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: substituí time.sleep(_TICK_INTERVAL * 3) por _wait_until(lambda: "4,120,000 lines" in buf.getvalue()).

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Verbose Test - tests/test_run_hcat_cmd.py, test_brain_rejection_recovers_with_the_long_form_client_flag.
Minhas principais dificuldades na remoção do code smell são: o teste era longo por causa do comando --brain-* grande e do setup de Popen; movi tudo pra uma fixture e separei os asserts em dois testes (comandos × estado final).
Usei o CATSPY para: Verbose Test.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: criei a @pytest.fixture brain_longform_recovery e dividi em 2 testes focados.

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Verbose Test - tests/test_run_hcat_cmd.py, test_brain_rejection_retries_at_most_once.
Minhas principais dificuldades na remoção do code smell são: método longo por causa do comentário extenso e do setup de Popen; movi o setup pra fixture e separei os asserts.
Usei o CATSPY para: Verbose Test.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: criei a @pytest.fixture brain_retry_once e dividi em 2 testes focados (nº de invocações × brain desativado).



UNIVERSIDADE
FEDERAL DO CEARÁ

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Verbose Test - tests/test_smart_mask.py, test_hcatSmartMask_reports_a_slot_exhausted_template_instead_of_dropping_it_quietly.
Minhas principais dificuldades na remoção do code smell são: longo por causa da montagem dos 9 charsets; movi a preparação pra uma fixture e separei a saída e o “não lançou hashcat” em dois testes.
Usei o CATSPY para: Verbose Test.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: criei a @pytest.fixture slot_exhausted_run e dividi em 2 testes focados.

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Verbose Test - tests/test_upload_cracked_hashes.py, test_upload_cracked_hashes_preserves_plaintext_whitespace.
Minhas principais dificuldades na remoção do code smell são: longo por docstring + setup de upload; movi a preparação pra fixture e separei “não pulou” de “chegou intacto”.
Usei o CATSPY para: Verbose Test.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: criei a @pytest.fixture padded_whitespace_upload e dividi em 2 testes focados.

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Verbose Test - tests/test_upload_cracked_hashes.py, test_upload_repairs_historical_double_encoded_ntlm_plaintext.
Minhas principais dificuldades na remoção do code smell são: tests/test_upload_cracked_hashes.py, test_upload_repairs_historical_double_encoded_ntlm_plaintext.
Usei o CATSPY para: Verbose Test.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: criei a @pytest.fixture double_encoded_repair_upload e dividi em 2 testes focados.

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Verbose Test - tests/test_upload_cracked_hashes.py, test_upload_cracked_hashes_preserves_plaintext_whitespace.
Minhas principais dificuldades na remoção do code smell são: longo por docstring + setup de upload; movi a preparação pra fixture e separei “não pulou” de “chegou intacto”.

Usei o CATSPY para: Verbose Test.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: criei a @pytest.fixture padded_whitespace_upload e dividi em 2 testes focados.

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Exception Handling - tests/test_api_downloads.py, test_keyboardinterrupt_cleanup.
Minhas principais dificuldades na remoção do code smell são: o teste capturava o KeyboardInterrupt na mão com try/except e uma flag booleana; precisei garantir que a verificação da limpeza do arquivo .part continuasse valendo depois de trocar pelo mecanismo do framework.
Usei o CATSPY para: Exception Handling.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: troquei o bloco try/except manual por with pytest.raises(Keyb)oardInterrupt e removi a variável de flag.

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Assertion Roulette - tests/test_api_downloads.py, lote inicial de testes (ex.: test_daemon_starts_with_expected_args, test_list_parses_rows, test_wait_for_all_reports_progress).
Minhas principais dificuldades na remoção do code smell são: vários asserts no mesmo teste sem indicar o que cada um valida; a dificuldade foi entender o que cada asserção checava para escrever uma mensagem fiel ao propósito.
Usei o CATSPY para: Assertion Roulette.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: adicionei uma mensagem explicativa (em inglês) a cada asserção.

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Assertion Roulette - tests/test_api_downloads.py, lote inicial de testes (continuação).
Minhas principais dificuldades na remoção do code smell são: distinguir, em testes com muitos asserts parecidos, a condição exata de cada um para que a mensagem não fique genérica.
Usei o CATSPY para: Assertion Roulette.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: adicionei uma mensagem explicativa a cada asserção.

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Assertion



UNIVERSIDADE
FEDERAL DO CEARÁ

Roulette - tests/test_api_downloads.py, test_404_on_primary_falls_back_to_official_alternate_url.
Minhas principais dificuldades na remoção do code smell são: o teste valida duas URLs diferentes (primária e fallback) em sequência; precisei deixar claro na mensagem qual URL cada assert verifica.
Usei o CATSPY para: Assertion Roulette.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: adicionei uma mensagem explicativa a cada asserção.

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Assertion Roulette - tests/test_api_downloads.py, test_successful_download.
Minhas principais dificuldades na remoção do code smell são: há duas versões desse teste (classes diferentes); garantir que ambas recebessem mensagens coerentes com o que cada uma baixa.
Usei o CATSPY para: Assertion Roulette.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: adicionei uma mensagem explicativa a cada asserção.

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Assertion Roulette - tests/test_api_downloads.py, test_untrusted_url_falls_back_to_hashmob_host.
Minhas principais dificuldades na remoção do code smell são: o teste é de segurança (header de api-key não pode vazar para host não confiável); as mensagens precisaram refletir essa intenção.
Usei o CATSPY para: Assertion Roulette.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: adicionei uma mensagem explicativa a cada asserção.

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Assertion Roulette - tests/test_api_downloads.py, test_retries_through_backoff_on_429.
Minhas principais dificuldades na remoção do code smell são: diferenciar nas mensagens os asserts de resultado, de contagem de chamadas e de backoff.
Usei o CATSPY para: Assertion Roulette.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: adicionei uma mensagem explicativa a cada asserção.

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Assertion Roulette - tests/test_api_downloads.py, test_happy_path.
Minhas principais dificuldades na remoção do code smell são: o teste mistura asserts de conteúdo do arquivo e de chamada do mock; escrever mensagens que separem os dois.
Usei o CATSPY para: Assertion Roulette.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: adicionei uma mensagem explicativa a cada asserção.

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Assertion Roulette - tests/test_api_downloads.py, test_gives_up_after_max_attempts.
Minhas principais dificuldades na remoção do code smell são: explicar nos asserts a lógica de "dorme entre tentativas, mas não após a última".
Usei o CATSPY para: Assertion Roulette.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: adicionei uma mensagem explicativa a cada asserção.

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Assertion Roulette - tests/test_api_downloads.py, test_gives_up_after_max_attempts (ajuste do commit anterior).
Minhas principais dificuldades na remoção do code smell são: revisar/complementar as mensagens do mesmo teste para cobrir todos os asserts.
Usei o CATSPY para: Assertion Roulette.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: adicionei/ajustei a mensagem explicativa nas asserções restantes.

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Assertion Roulette - tests/test_api_downloads.py, test_creates_cache_when_missing.
Minhas principais dificuldades na remoção do code smell são: deixar claro nas mensagens a diferença entre "o cache foi criado" e "o cache tem o conteúdo certo".
Usei o CATSPY para: Assertion Roulette.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: adicionei uma mensagem explicativa a cada asserção.

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Verbose Test -

tests/test_api_downloads.py, test_rate_coerced_to_float.
Minhas principais dificuldades na remoção do code smell são: longo por causa de três dicionários de resposta repetidos inline; movi a construção para helpers sem perder os casos de rate string/inválido/nulo.
Usei o CATSPY para: Verbose Test.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: usei os helpers _weakpass_entry/_json_response para reduzir o teste para menos de 30 linhas.

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Verbose Test - tests/test_api_downloads.py, test_falsy_ids_not_deduplicated.
Minhas principais dificuldades na remoção do code smell são: quatro entradas grandes com ids "falsy" (None, "") repetidas; encurtar mantendo a verificação de que nenhuma é deduplicada.
Usei o CATSPY para: Verbose Test.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: usei os helpers _weakpass_entry/_json_response para reduzir o teste para menos de 30 linhas.

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Conditional Test Logic - tests/test_api_downloads.py, test_three_pages_fetched_and_returned_in_order.
Minhas principais dificuldades na remoção do code smell são: o mock escolhia a resposta por página com if/elif; precisei mapear cada página para sua resposta sem alterar o comportamento.
Usei o CATSPY para: Conditional Test Logic.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: troquei o if/elif por um dicionário de respostas indexado pela página (pages.get(page, ...)).

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Conditional Test Logic - tests/test_api_downloads.py, test_same_id_on_two_pages_kept_once.
Minhas principais dificuldades na remoção do code smell são: preservar a semântica de "primeira ocorrência vence" ao transformar o if/elif em dicionário.
Usei o CATSPY para: Conditional Test Logic.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: troquei o if/elif por um dicionário de respostas indexado pela página.

UNIVERSIDADE FEDERAL DO CEARÁ

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Conditional Test Logic - tests/test_api_downloads.py, test_total_pages_caps_at_last_page.
Minhas principais dificuldades na remoção do code smell são: havia condicional em dois lugares — no mock e no laço for/if dos asserts; precisei eliminar os dois sem sobrar nenhuma estrutura de controle.
Usei o CATSPY para: Conditional Test Logic.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: troquei o if/elif por dicionário de respostas e substituí o laço dos asserts por uma construção sem for.

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Conditional Test Logic - tests/test_api_downloads.py, test_real_fetch_integration_all_pages_ok.
Minhas principais dificuldades na remoção do code smell são: é um teste de integração com callback real; garantir que a troca do if/elif por dicionário não mudasse o fluxo de cache/download.
Usei o CATSPY para: Conditional Test Logic.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: troquei o if/elif por um dicionário de respostas indexado pela página.

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Conditional Test Logic - tests/test_api_downloads.py, test_page2_of3_fails_returns_pages1and3.
Minhas principais dificuldades na remoção do code smell são: a página 2 falha no raise_for_status; precisei colocar a resposta que falha dentro do dicionário para não deixar um if page == 2 residual.
Usei o CATSPY para: Conditional Test Logic.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: criei o helper _failing_resp(exc) e incluí a resposta que falha como mais uma chave do dicionário.

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Conditional Test Logic - tests/test_api_downloads.py, test_page2_json_decode_error_returns_pages1and3.
Minhas principais dificuldades na remoção do code smell são: a página 2 quebra no .json() com ValueError; representar esse caso de erro como um valor do dicionário em vez de um ramo condicional.
Usei o CATSPY para: Conditional Test Logic.

Eu estou usando os seguintes métodos de refatoração para remover o code smell: criei o helper _json_decode_error_resp() e incluí a resposta com erro de JSON como chave do dicionário.

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Verbose Test - tests/test_api_downloads.py, test_tier_mapping
Minhas principais dificuldades na remoção do code smell são: longo por causa de três dicionários de resposta grandes inline (tiers S, b minúsculo e None); encurtar mantendo a verificação do mapeamento tier→rank.
Usei o CATSPY para: Verbose Test.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: usei os helpers _weakpass_entry/_json_response para reduzir o teste para menos de 30 linhas.

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Unknown Test - tests/test_api_downloads.py, test_startup_timeout_raises.
inhas principais dificuldades na remoção do code smell são: o teste verificava o RuntimeError do timeout com with pytest.raises(...), mas sem um assert explícito, então o TEMPY o lia como "sem asserção"; precisei capturar a exceção e verificar sua mensagem.
Usei o CATSPY para: Unknown Test.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: capturei a exceção com as exc_info e adicionei assert "Transmission daemon failed" in str(exc_info.value).

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Unknown Test - tests/test_api_downloads.py, test_add_raises_when_torrent_not_added.
Minhas principais dificuldades na remoção do code smell são: o teste checava o RuntimeError de "torrent não adicionado" apenas com pytest.raises, sem asserção explícita sobre o resultado.
Usei o CATSPY para: Unknown Test.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: capturei a exceção com as exc_info e adicionei assert isinstance(exc_info.value, RuntimeError).

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Unknown Test - tests/test_api_downloads.py, test_wait_for_all_keyboard_interrupt_propagates.
Minhas principais dificuldades na remoção do code smell são: o teste confirmava que o KeyboardInterrupt se propaga de wait_for_all, mas sem asserção explícita.
Usei o CATSPY para: Unknown Test.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: capturei a exceção com as exc_info e adicionei assert isinstance(exc_info.value, KeyboardInterrupt).

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Unknown Test - tests/test_api_downloads.py, test_keyboard_interrupt_propagates.
Minhas principais dificuldades na remoção do code smell são: o teste verificava a propagação do KeyboardInterrupt em run_torrent_session só com pytest.raises, sem asserção explícita.
Usei o CATSPY para: Unknown Test.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: capturei a exceção com as exc_info e adicionei assert isinstance(exc_info.value, KeyboardInterrupt).

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Unknown Test - tests/test_api_downloads.py, test_invalid_variant_raises_value_error.
Minhas principais dificuldades na remoção do code smell são: o teste confirmava o ValueError de uma variante inválida apenas com pytest.raises, sem asserção explícita.
Usei o CATSPY para: Unknown Test.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: capturei a exceção com as exc_info e adicionei assert isinstance(exc_info.value, ValueError).

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Unknown Test - tests/test_api_downloads.py, test_non_429_exception_reraises.
Minhas principais dificuldades na remoção do code smell são: o teste confirmava que uma exceção não-429 é re-lançada, mas sem asserção explícita sobre a mensagem.
Usei o CATSPY para: Unknown Test.

Eu estou usando os seguintes métodos de refatoração para remover o code smell: capturei a exceção com as exc_info e adicionei assert "not a 429" in str(exc_info.value).

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Conditional Test Logic - tests/test_api_downloads.py, test_real_fetch_integration_page2_fails.
Minhas principais dificuldades na remoção do code smell são: o mock decidia a resposta por página com if/elif e a página 2 falhava; precisei colocar a resposta que falha dentro do dicionário para não deixar um if residual.
Usei o CATSPY para: Conditional Test Logic.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: criei o helper _failing_resp(exc) e troquei o if/elif por um dicionário de respostas indexado pela página.

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Conditional Test Logic - tests/test_api_downloads.py, test_multiple_candidates_tries_each.
Minhas principais dificuldades na remoção do code smell são: o mock decidia 404 vs sucesso por substring da URL com if/elif; precisei eliminar o condicional sem perder os dois ramos.
Usei o CATSPY para: Conditional Test Logic.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: troquei o if/elif por um dicionário indexado por condição ({True: _not_found(), False: _resolved()}[".7z" in url]), com funções auxiliares para as respostas.

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Conditional Test Logic - tests/test_api_downloads.py, test_unresolvable_bare_name_not_found_message.
Minhas principais dificuldades na remoção do code smell são: havia dois condicionais — o if no mock e o for nos asserts; precisei eliminar os dois sem deixar nenhuma estrutura de controle.
Usei o CATSPY para: Conditional Test Logic.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: troquei o if do mock por um dicionário indexado por condição e o laço dos asserts por "weakpass.com/files/" not in "\n".join(seen_urls).